Defect workflow, as dwf.py runs it

One column per phase, top to bottom; each diamond is a decision and every branch out of it is labelled. One command runs every phase in order: `python3 dwf.py <name>.dwf`; each phase keeps its results.json (read by the later phases) and summary.txt in its folder.

start / end · step done by dwf.py · CASTEP jobs (supercell_convergence.py) · MACE (charge-blind; runs where dwf.py runs) · ShakeNBreak · your action · decision · input · B on to the next phase

A Host reference

A_host/ · once per host

project file <name>.dwf, host .cell, .param, jobscript

converge true?
- no → use the .param and k-points as given
- yes

A1 converge
k-point grids at a cheap cut-off, then converge_cutoffs

all within converge_tol of the best?
- no → add higher cut-offs or finer grids, rerun
- yes

A2.1 relax
lattice and positions, with symmetry → relaxed cell, e_bulk

A2.2 bands
Γ-centred grid → VBM, CBM, band gap

dielectric set in the project file?
- yes → use that value
- no

A2.3 dielectric
DFPT phonon+efield at Γ → ε₀, ε∞

%block chemical_potentials given?
- yes → use those limits
- no

phases given? (%block phases or doped + MP)
- no → no limits: E2 then leaves out the μ terms
- yes

A3.1 phases
relax the competing phases

host stable against its phases?
- no → stop: check the phases and their settings
- yes

A3.2 limits
μ at each corner (pymatgen); each dopant: its most stable phase

→ B

B Candidates and cell size

B_candidates/ · once per host

A

%block defects given? (defect known)
- yes → **B1 your defects** the lines of %block defects; all: every arrangement
- no

B1 generate
vacancies, dopant substitutions, (antisites, Voronoi interstitials), dopant + native pairs up to pair_max_distance; duplicates removed by symmetry

supercell : n given?
- yes → skip B2–B4
- no

mace and scan?
- no → no cap
- yes

B2 MACE size scan
relaxation energy against size; caps the series where it levels off

B3 CASTEP size series
supercell_convergence.py: MACE pre-relaxation, then CASTEP, size by size

|ΔE_f*| < series_tol between two sizes?
- no → sizes left?
  - yes → next size
  - no → stop: add sizes, or set supercell
- yes

finetune true?
- yes → **B4 fine-tuning data** train.xyz from the series; finetune.sh → --until B, run it; set finetuned_model
- no

B5 choose
the cell used from here on (the smaller converged size)

→ C

C Screening and ground states

C_screen/ · every candidate, then survivors

B

C1 build and pre-relax
every candidate in the B5 cell; MACE relaxation (positions only)

triage true?
- yes → drop those more than triage_window above their group's lowest
- no

C2 screen
Γ point, screen_charges × starting spins (low, +2), pristine cell too; no restarts: a failure is flagged

C3: on the defect hull (hull_margin), not a duplicate, filters met?
- no → dropped (why: summary.txt)
- yes

C3b: first ZPL estimate in zpl_window?
- no → dropped (why: summary.txt)
- yes (or no window set)

C4 distort (ShakeNBreak)
bond distortions of the n nearest neighbours + rattle, each charge state

C5 relax the distortions
C2 settings

C6 rank
per charge state; identical structures merged

a distortion wins by distortion_min_gain and others lack it?
- yes → **share** relax it in those charge states; at most share_rounds rounds
- no

keep the lowest keep_structures per charge state

→ D

D Production relaxations

D_production/ · survivors × charge states

C

freeze_radius > 0? (D2, untested idea)
- yes → atoms beyond it held fixed (ionic constraints)
- no

D1 production relaxations
A1 settings + production_overrides, fixed cell, screened spin; plus the pristine supercell

a job ran out of time?
- yes → resubmitted, carrying on from its .check (max_restarts)
- no

a job failed?
- yes → flagged and left out of E; fix it, rerun with --resubmit-failed
- no

→ E

E Thermodynamics

E_thermo/ · per defect

D

E1 corrections
Lany–Zunger (or Makov–Payne) with ε₀ from A2.3

E2 formation energies
at each μ limit; transition levels; formation.png

fermi_temperature set?
- yes → self-consistent Fermi level, concentrations; quench_temperature: frozen in, then redistributed
- no

E3: a deep level, or an internal transition found in C?
- no → end: formation energies and levels
- yes

→ F

F Optical lineshape

not scripted yet · selected defects

E

F1 two geometries
ground and excited state (charge states or ΔSCF)

F2 training set
every D1 step + displaced single points around the minimum

F3 fine-tune MACE
one model per state

F4: reproduces CASTEP forces?
- no → more displaced single points
- yes

F5 phonons and lineshape
Huang–Rhys factors, PL and absorption spectra

end: spectra

Every CASTEP step: the job runner

supercell_convergence.py batch mode, for each step:
- one dry run per distinct input, then a plan per job: nodes, memory, partition, walltime
- at most max_jobs queued at once
- out of time: carried on from the .check (max_restarts; 0 for screening)
- tolerate_failures: a failed job is flagged and the others carry on (screening, production)
- a job whose inputs changed since it ran stops the run, so results from two settings never mix
- touch <workdir>/STOP stops it; jobs keep running

Checkpoints and reruns

<workdir>/state.json records each finished step with a fingerprint of its settings, files and earlier results.
- stopped (STOP, Ctrl-C, --no-wait, a crash)? Rerun the same command: finished steps are skipped, submitted and finished jobs reused
- changed a setting? The steps that use it, and those after them, run again; unchanged jobs are reused
- --redo STEP: that step from scratch (old files kept)
- --status: what has finished; --until B: stop after B
- --prepare-only: write the inputs, submit nothing

Phase F is not scripted yet

E3 lists the defects worth a lineshape (for_F in E_thermo/results.json); each D1 relaxation's .castep (every step) is training data for F2. Fine-tuning and phonons: MACE and lineshape_tools (Turiansky et al.).

Charge-aware MACE: not in the workflow yet

Every MACE step in this chart (green) is charge-blind: the model sees no charge and no spin. It only prepares structures (B2, B3, C1) and never ranks charge states, so CASTEP does all the searching in C and D, and F needs one fine-tuned model per charge state.

A charge-aware MACE (Wang, Mosquera-Lois and Walsh, arXiv:2603.05238, March 2026):
- a learnable total-charge embedding in MACE (v0.3.15): one model covers all five charge states of a Se vacancy in Sb_2Se_3
- multi-fidelity: PBE relaxations, plus HSE06 single points on about 10 % of them
- transition levels within 0.02 eV of HSE06; exploring the energy surface about three orders of magnitude cheaper
- models with explicit charges (MACE-POLAR, MACE-LES) did worse there, and the molecular MACE-POLAR-1 missed the DFT ground states

What would stay. CASTEP makes the training data and has the last word: gap states, spin and excited states still need it. A model trained on one defect is reliable only near the structures it has seen, so a minimum it finds far from them is a candidate to check in CASTEP, not a result.

Status. One preprint and one test material; its data and trained weights "will be made available upon publication". It used VASP (PAW); with CASTEP, the PBE and hybrid runs would need the same pseudopotentials and settings.

Now: dwf.py

C2 screen — cheap CASTEP pass, every candidate, charge state and starting spin
↓
C4–C6 ground-state search — distortions relaxed in CASTEP, in every charge state
↓
D1 production — CASTEP relaxations of the best structures, every charge state
↓
E2 levels — at the D1 functional
↓
F2–F4 phonons (not scripted) — one fine-tuned, charge-blind MACE per charge state

With a charge-aware, multi-fidelity MACE

training data — a few cheap CASTEP relaxations and distortions per charge state
↓
hybrid single points — on about 10 % of those structures
↓
one charge-aware MACE per defect — every charge state, trained on both fidelities
↓
search with the model — distortions and metastable states in every charge state, levels at hybrid level, phonons: replaces C4–C6, most of D1 and F2–F4
↓
CASTEP checks — final structures; gap states, spin, ΔSCF